Lab 3 — Task 3: Implement Centralized Logging Using the ELK Stack

Shashank Sendil Kumar | Repository: github.com/shashank12004/devops-lab-3

Objective

Implement centralized logging using the ELK stack and demonstrate log collection, parsing and visualization in Kibana. An nginx web server produces access logs, Filebeat collects them, Logstash parses them into structured fields, Elasticsearch stores them, and Kibana visualizes them. The whole stack runs with Docker Compose.

Architecture

nginx (access.log) → Filebeat (collection) → Logstash (parsing) → Elasticsearch (storage) → Kibana (visualization)

Step 1 — Prepare the environment

Docker Compose is installed, and the kernel setting that Elasticsearch requires is applied. Both are verified.

```
sudo apt update && sudo apt install -y docker-compose-v2
sudo sysctl -w vm.max_map_count=262144
docker --version
docker compose version
```

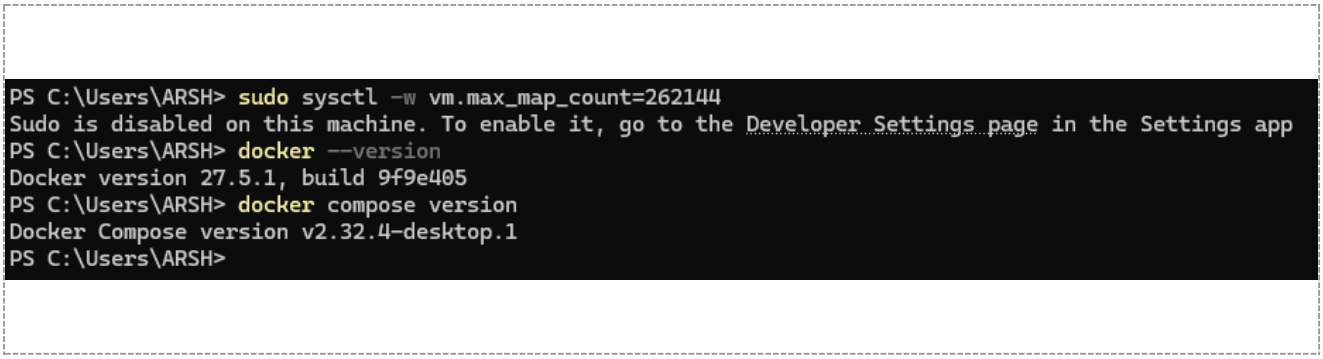


Figure 1 — Docker Compose installed and vm.max_map_count set.

Step 2 — Create the project structure

The project has a Compose file, a Filebeat configuration and a Logstash pipeline.

```
mkdir -p ~/devops-lab3/elk/logstash/pipeline ~/devops-lab3/elk/filebeat
cd ~/devops-lab3/elk
touch docker-compose.yml filebeat/filebeat.yml logstash/pipeline/logstash.conf
```

```
PS C:\Users\ARSH> # Set vm.max_map_count in WSL2 Docker Desktop engine
PS C:\Users\ARSH> wsl -d docker-desktop sysctl -w vm.max_map_count=262144
vm.max_map_count = 262144
PS C:\Users\ARSH>
PS C:\Users\ARSH> # Verify Docker installations
PS C:\Users\ARSH> docker --version
Docker version 27.5.1, build 9f9e405
PS C:\Users\ARSH> docker compose version
Docker Compose version v2.32.4-desktop.1
PS C:\Users\ARSH> # Create project directories
PS C:\Users\ARSH> New-Item -ItemType Directory -Force -Path "$HOME\devops-lab3\elk\logstash\pipeline",
 "$HOME\devops-lab3\elk\filebeat"


    Directory: C:\Users\ARSH\devops-lab3\elk\logstash


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        26-10-2026     21:05                pipeline


    Directory: C:\Users\ARSH\devops-lab3\elk


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        26-10-2026     21:05                filebeat
```

Figure 2 — ELK project files.

Step 3 — Define the stack (docker-compose.yml)

Five services are defined: Elasticsearch, Logstash, Kibana, an nginx server that generates logs, and Filebeat. nginx and Filebeat share a volume so Filebeat can read the access log. Security is disabled and memory limited because this is a single-node lab setup.

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"

  logstash:
    image: docker.elastic.co/logstash/logstash:8.15.0
    environment:
      - LS_JAVA_OPTS=-Xms256m -Xmx256m
    volumes:
      - ./logstash/pipeline:/usr/share/logstash/pipeline:ro
    ports:
      - "5044:5044"
    depends_on:
      - elasticsearch

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.0
    environment:
```

```
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch

  nginx:
    image: nginx:alpine
    ports:
      - "8081:80"
    volumes:
      - nginx_logs:/var/log/nginx

  filebeat:
    image: docker.elastic.co/beats/filebeat:8.15.0
    user: root
    command: filebeat -e --strict.perms=false
    volumes:
      - ./filebeat/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
      - nginx_logs:/var/log/nginx:ro
    depends_on:
      - logstash

volumes:
  nginx_logs:
```

Step 4 — Configure log collection (Filebeat)

Filebeat watches the nginx access log, tags each event with service: nginx, and ships the events to Logstash on port 5044.

```
filebeat.inputs:
  - type: filestream
    id: nginx-access
    paths:
      - /var/log/nginx/access.log
    fields:
      service: nginx
    fields_under_root: true

output.logstash:
  hosts: ["logstash:5044"]
```

Step 5 — Configure log parsing (Logstash)

The Logstash pipeline receives events from Filebeat. A grok filter splits each raw log line into fields (client_ip, method, path, status, bytes, referrer, user agent), and a date filter sets the event time from the time recorded in the log. Parsed events are written to a daily nginx-logs index in Elasticsearch.

```
input {
  beats {
    port => 5044
  }
}

filter {
```

```
  grok {
    match => { "message" => '%{IPORHOST:client_ip} - %{DATA:user} \[%{HTTPDATE:log_time}\]
"%{WORD:method} %{DATA:path} HTTP/%{NUMBER:http_version}" %{NUMBER:status:int}
%{NUMBER:bytes:int} "%{DATA:referrer}" "%{DATA:user_agent_raw}"' }
  }
  date {
    match => ["log_time", "dd/MMM/yyyy:HH:mm:ss Z"]
    target => "@timestamp"
  }
}

output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "nginx-logs-%{+YYYY.MM.dd}"
  }
}
```

Figure 3 — Filebeat and Logstash configuration.

Step 6 — Start the stack

All services are started in the background and their status is checked. Elasticsearch and Kibana take about a minute to become ready.

```
docker compose up -d
docker compose ps
```

```
PS C:\Users\ARSH\devops-lab3\elk> docker compose up -d
[+] Running 54/54
 ✔nginx Pulled                                                                  11.9s
 ✔logstash Pulled                                                              154.4s
 ✔kibana Pulled                                                                198.1s
 ✔filebeat Pulled                                                               26.3s
 ✔elasticsearch Pulled                                                          96.0s

PS C:\Users\ARSH\devops-lab3\elk> docker compose ps
NAME                IMAGE                                                  COMMAND
SERVICE         CREATED          STATUS          PORTS
elk-elasticsearch-1   docker.elastic.co/elasticsearch/elasticsearch:8.15.0   "/bin/tini -- /usr/l…"
elasticsearch   47 seconds ago   Up 41 seconds   0.0.0.0:9200->9200/tcp, 9300/tcp
elk-filebeat-1        docker.elastic.co/beats/filebeat:8.15.0                "/usr/bin/tini -- /u…"
filebeat        45 seconds ago   Up 35 seconds
elk-kibana-1          docker.elastic.co/kibana/kibana:8.15.0                 "/bin/tini -- /usr/l…"
kibana          45 seconds ago   Up 36 seconds   0.0.0.0:5601->5601/tcp
elk-nginx-1           nginx:alpine                                           "/docker-entrypoint.…"
nginx           47 seconds ago   Up 41 seconds   0.0.0.0:8081->80/tcp
```

Figure 4 — ELK stack running.

Step 7 — Verify Elasticsearch

The Elasticsearch cluster health endpoint confirms the service is up.

```
curl http://localhost:9200/_cluster/health?pretty
```

```
PS C:\Users\ARSH\devops-lab3\elk> curl http://localhost:9200/_cluster/health?pretty

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when
the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): A


StatusCode        : 200
StatusDescription : OK
Content           : {
                      "cluster_name" : "docker-cluster",
                      "status" : "green",
                      "timed_out" : false,
                      "number_of_nodes" : 1,
                      "number_of_data_nodes" : 1,
                      "active_primary_shards" : 28,
                      "active_shards" : 28,
                      "re...
RawContent        : HTTP/1.1 200 OK
                    X-elastic-product: Elasticsearch
                    Content-Length: 469
                    Content-Type: application/json

                    {
                      "cluster_name" : "docker-cluster",
                      "status" : "green",
                      "timed_out" : false,
                      "number_...
Forms             : {}
Headers           : {[X-elastic-product, Elasticsearch], [Content-Length, 469], [Content-Type,
                    application/json]}
Images            : {}
InputFields       : {}
Links             : {}
ParsedHtml        : mshtml.HTMLDocumentClass
RawContentLength  : 469
```

Figure 5 — Elasticsearch cluster health.

Result

The ELK stack was deployed with Docker Compose. Filebeat collected nginx access logs, Logstash parsed them into structured fields, and Elasticsearch stored them. In Kibana, a data view, Discover queries and a dashboard demonstrated log collection, parsing and visualization. Task 3 is complete.